

LAB DAY 19: XÂY DỰNG HỆ THỐNG GRAPH RAG VỚI TECH COMPANY CORPUS

1. MỤC TIÊU BÀI HỌC

- Hiểu rõ quy trình trích xuất thực thể (Entity Extraction) và quan hệ (Relation Extraction) từ văn bản thô.
- Làm quen với các thư viện quản lý đồ thị: **NetworkX**, **Neo4j** và framework mã nguồn mở **NodeRAG**.
- Xây dựng hoàn chỉnh một pipeline GraphRAG: từ lập chỉ mục (Indexing) đến truy vấn đa bước (Multi-hop Querying).
- Đánh giá sự khác biệt về độ chính xác giữa Flat RAG và GraphRAG.

2. PHẦN 1: NGHIÊN CỨU VÀ CHUẨN BỊ (RESEARCH)

Trước khi bắt đầu code, sinh viên cần tìm hiểu các khái niệm và công cụ sau:

2.1. Quy trình xử lý dữ liệu đồ thị

Sinh viên cần trả lời được các câu hỏi:

1. **Entity Extraction:** Làm sao để LLM phân biệt được đâu là thực thể (Node) và đâu là thuộc tính?
2. **Graph Construction:** Tại sao việc khử trùng lặp (Deduplication) lại quan trọng trong đồ thị?
3. **Query Answering:** Sự khác biệt giữa duyệt đồ thị theo chiều rộng (BFS) và tìm kiếm vector thông thường là gì?

2.2. Tìm hiểu công cụ

- **NetworkX:** Thư viện Python dùng để nghiên cứu các mạng lưới phức tạp. Phù hợp cho việc tạo prototype nhanh.
- **Neo4j:** Cơ sở dữ liệu đồ thị chuẩn công nghiệp, sử dụng ngôn ngữ truy vấn Cypher.
- **NodeRAG:** Một framework mã nguồn mở xây dựng trên nền NetworkX, giúp đơn giản hóa việc tích hợp GraphRAG vào ứng dụng Python.

3. PHẦN 2: ENVIRONMENT SETUP

Mở terminal hoặc command prompt và cài đặt các thư viện cần thiết:

```
Bash
```

```
# Cài đặt các thư viện cơ bản cho xử lý ngôn ngữ và đồ thị
pip install networkx matplotlib neo4j openai pandas
```

```
# Cài đặt NodeRAG framework
pip install noderag
```

```
# Nếu sử dụng LangChain để hỗ trợ pipeline
pip install langchain langchain-openai
```

Lưu ý: Đối với Neo4j, sinh viên nên sử dụng **Neo4j Desktop** hoặc chạy qua **Docker** để có giao diện trực quan hóa (Bloom/Browser).

4. PHẦN 3: HƯỚNG DẪN THỰC HIỆN TỪNG BƯỚC

Bước 1: Trích xuất thực thể và quan hệ (Indexing)

Sử dụng LLM để đọc bộ dữ liệu "Tech Company Corpus" và chuyển đổi thành các bộ ba (Triples).

- **Input:** "OpenAI được thành lập bởi Sam Altman và Elon Musk vào năm 2015."
- **Output (Triples):**
 - (OpenAI, FOUNDED_BY, Sam Altman)
 - (OpenAI, FOUNDED_BY, Elon Musk)
 - (OpenAI, FOUNDED_IN, 2015)

Bước 2: Xây dựng đồ thị (Construction)

Sinh viên thực hiện đẩy dữ liệu vào một trong ba công cụ sau:

- **Lựa chọn A (NetworkX):** Phù hợp để chạy offline trong Notebook.
- **Lựa chọn B (Neo4j):** Khuyến dùng nếu muốn trực quan hóa các mối liên kết bằng mắt thường.
- **Lựa chọn C (NodeRAG):** Sử dụng nếu muốn một giải pháp trọn gói (all-in-one) đã được tối ưu sẵn logic tìm kiếm.

Bước 3: Thực thi truy vấn (Querying)

Viết hàm xử lý truy vấn theo logic:

- 1. Nhận câu hỏi từ người dùng.
- 2. Trích xuất thực thể chính trong câu hỏi (ví dụ: "Google").
- 3. Tìm node tương ứng trong đồ thị và duyệt (traverse) các node lân cận trong phạm vi 2-hop.
- 4. Gộp các thông tin tìm được thành một đoạn văn (Textualization) và gửi cho LLM.

Bước 4: So sánh và Đánh giá (Evaluation)

Sinh viên chạy thử 5 câu hỏi phức tạp trên cả hai hệ thống:

- 1. **Flat RAG:** Chỉ dùng ChromaDB/Faiss.
- 2. **GraphRAG:** Dùng đồ thị vừa xây dựng.
 - *Yêu cầu:* Ghi lại các trường hợp Flat RAG bị ảo giác nhưng GraphRAG trả lời đúng.

5. ĐỀ XUẤT CÔNG CỤ (RECOMMENDATIONS)

Mục tiêu	Tool gợi ý	Lý do
Dễ bắt đầu	NodeRAG	Tích hợp sẵn logic GraphRAG, không cần cấu hình database phức tạp
Trực quan hóa tốt nhất	Neo4j	Giao diện đồ họa giúp "thấy" được tri thức đang được kết nối như thế nào.
Nghiên cứu thuật toán	NetworkX	Cho phép can thiệp sâu vào các thuật toán toán học của đồ thị.

6. DELIVERABLES

Sinh viên nộp báo cáo bao gồm:

1. Mã nguồn (File .py hoặc .ipynb).
2. Ảnh chụp màn hình đồ thị tri thức đã xây dựng (từ Neo4j hoặc Matplotlib).
3. Bảng so sánh kết quả 20 câu hỏi benchmark giữa Flat RAG và GraphRAG.
4. Phân tích ngắn gọn về chi phí (Token usage, time) khi xây dựng đồ thị.